

Functional programming & rectangling

Lecture 10

Dr. Colin Rundel

Functional programming

Functions as objects

Functions are first-class objects in both languages - they can be assigned to names, stored in lists and dicts, and passed to other functions.

```
1 f = function(x) x^2
2 g = f
3 g(3)
```

R

[1] 9

```
1 l = list(sq = f, root = sqrt)
2 l$sq(3)
```

R

[1] 9

```
1 l[[2]](9)
```

R

[1] 3

```
1 l[2](3)
```

R

Error:
! attempt to apply non-function

```
1 def f(x):
2     return x**2
3
4 g = f
5 g(3)
```

py

9

```
1 import math
2 d = {"sq": f, "root": math.sqrt}
3 d["sq"](3)
```

py

9

```
1 d["root"](9)
```

py

3.0

Functions as arguments

A function that takes a function as an argument is a higher-order function. Note that the function object is passed (`sum`), not a call to it (`sum()`).

```
1 do_calc = function(v, func) {  
2   func(v)  
3 }  
4 do_calc(1:3, sum)
```

R

[1] 6

```
1 do_calc(1:3, mean)
```

R

[1] 2

```
1 def do_calc(v, func):  
2   return func(v)  
3  
4 do_calc([1, 2, 3], sum)
```

py

6

```
1 do_calc([1, 2, 3], max)
```

py

3

Many built-in functions work this way,

```
1 integrate(dnorm, -1.96, 1.96)
```

R

0.9500042 with absolute error < 1e-11

```
1 sorted(["bb", "a", "ccc"], key=len)
```

py

['a', 'bb', 'ccc']

Anonymous functions

Short single-use functions can be written inline without a name: `function(x)` or `\(x)` (v4.1+) in R, `lambda` in Python - a single expression, whose value is returned.

```
1 (\(x) x^2)(4)
```

R

```
[1] 16
```

```
1 integrate(\(x) sin(x)^2, 0, pi)
```

R

```
1.570796 with absolute error < 1.7e-14
```

```
1 (lambda x: x**2)(4)
```

py

```
16
```

```
1 sorted(  
2     ["Bob", "alice", "Carol"],  
3     key=lambda s: s.lower()  
4 )
```

py

```
['alice', 'Bob', 'Carol']
```

PEP 8 says not to assign a `lambda` to a name (`sq = lambda x: x**2`); write a `def` instead. Both shorthands are meant for

Partial application

Partial application fixes some of a function's arguments, producing a new function with a reduced signature. A wrapper does this by hand; `partial()` does it directly, by name or by position.

```
1 pow = function(base, exp) base^exp
2 cube = partial(pow, exp = 3)
```

```
1 cube(2)
```

```
[1] 8
```

```
1 partial(pow, 2)(5)
```

```
[1] 32
```

```
1 partial(pow, ..., 3)(2)
```

```
[1] 8
```

```
1 from functools import partial, Placeholder
2 cube = partial(pow, exp=3)
```

```
1 cube(2)
```

```
8
```

```
1 partial(pow, 2)(5)
```

```
32
```

```
1 partial(pow, Placeholder, 3)(2)
```

```
8
```

Python's `pow(base, exp)` is built in, so it is defined here only in R. Unnamed arguments fill from the left; `... =` in purrr

Loops vs maps

Of course, someone has to write loops. It doesn't have to be you.

— Jenny Bryan

Most loops compute something per element and collect the results. A map does the looping and collecting, leaving only the computation - generally not faster, just clearer.

```
1 x = c(1, 4, 9)
2 res = numeric(length(x))
3 for (i in seq_along(x)) res[i] = sqrt(x[i])
4 res
```

```
[1] 1 2 3
```

```
1 lapply(x, sqrt) |> str()
```

```
List of 3
 $ : num 1
 $ : num 2
 $ : num 3
```

```
1 sapply(x, sqrt) |> str()
```

```
num [1:3] 1 2 3
```

```
1 x = [1, 4, 9]
2 res = []
3 for v in x:
4     res.append(v**0.5)
5 res
```

```
[1.0, 2.0, 3.0]
```

```
1 [v**0.5 for v in x]
```

```
[1.0, 2.0, 3.0]
```

```
1 list(map(lambda v: v**0.5, x))
```

```
[1.0, 2.0, 3.0]
```

lapply() and sapply()

`lapply(X, FUN, ...)` applies `FUN` to each element of `X` and always returns a list; extra arguments pass through `...`.

`sapply()` simplifies the result to a vector or matrix when it can (returned type depends on the data).

```
1 lapply(1:3, \(x) x^2) |> str()
```

```
List of 3
 $ : num 1
 $ : num 4
 $ : num 9
```

```
1 lapply(1:3, \(x, p) x^p, p = 3) |> str()
```

```
List of 3
 $ : num 1
 $ : num 8
 $ : num 27
```

```
1 sapply(1:3, \(x) x^2)
```

```
[1] 1 4 9
```

```
1 sapply(1:3, \(x) c(x, x^2))
```

```
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    1    4    9
```

```
1 sapply(1:3, seq) |> str()
```

```
List of 3
 $ : int 1
 $ : int [1:2] 1 2
 $ : int [1:3] 1 2 3
```



map()

`map(.x, .f, ...)` is `lapply()` with a consistent interface. The typed variants `map_lgl()`, `map_int()`, `map_dbl()`, and `map_chr()` return a vector of that type or fail. `walk()` exists for side effects.

```
1 map(1:3, \(x) x^2) |> str()
```

R

```
List of 3
 $ : num 1
 $ : num 4
 $ : num 9
```

```
1 map_chr(1:3, \(x) x^2)
```

R

```
Error in `map_chr()`:
i In index: 1.
Caused by error:
! Can't coerce from a number to a string.
```

```
1 map_dbl(1:3, \(x) x^2)
```

R

```
[1] 1 4 9
```

```
1 map_chr(1:3, \(x) paste0("n", x))
```

R

```
[1] "n1" "n2" "n3"
```

```
1 map_int(1:3, \(x) x / 2)
```

R

```
Error in `map_int()`:
i In index: 1.
Caused by error:
! Can't coerce from a number to an integer.
```

map2(), pmap(), and imap()

`map2()` iterates over two inputs in parallel, `pmap()` over any number (including the columns of a data frame, row by row), and `imap()` over elements and their names or positions. All have the same typed variants as `map()`.

```
1 map2_chr(letters[1:3], 1:3, paste0)
```

R

```
[1] "a1" "b2" "c3"
```

```
1 map2_dbl(1:3, 4:6, \(x, y) x * y)
```

R

```
[1] 4 10 18
```

```
1 pmap_dbl(  
2   list(1:3, 4:6, 7:9),  
3   \(x, y, z) x + y + z  
4 )
```

R

```
[1] 12 15 18
```

```
1 d = tibble(x = 1:2, y = c("a", "b"))  
2 pmap_chr(d, \(x, y) paste(x, y))
```

R

```
[1] "1 a" "2 b"
```

```
1 imap_chr(  
2   c(a = 1, b = 2),  
3   \(x, i) paste(i, x)  
4 )
```

R

```
      a      b  
"a 1" "b 2"
```

Extracting elements

Often `.f` just extracts a piece of each element, so `purrr` lets a name or position be used directly. `.default` is used when the path does not exist (the result would otherwise be `NULL`).

```
1 x = list(list(name = "a", tags = list("x", "y")),
2         list(name = "b"),
3         list(name = "c", tags = list("z")))
```

```
1 map_chr(x, "name")
```

```
[1] "a" "b" "c"
```

```
1 map(x, "tags") |> lengths()
```

```
[1] 2 0 1
```

```
1 map_chr(x, list("tags", 1))
```

```
Error in `map_chr()`:
i In index: 2.
Caused by error:
! Result must be length 1, not 0.
```

```
1 map_chr(
2   x, list("tags", 1), .default = NA
3 )
```

```
[1] "x" NA  "z"
```

keep() and discard()

`keep()` and `discard()` filter a vector or list by a predicate function, returning the elements where it is `TRUE` or `FALSE` respectively. The predicate takes the same forms as `.f` in `map()`, including the name shorthand.

```
1 keep(1:10, \(x) x %% 3 == 0)
```

R

```
[1] 3 6 9
```

```
1 discard(1:10, \(x) x %% 3 == 0)
```

R

```
[1] 1 2 4 5 7 8 10
```

```
1 x |>  
2   keep(\(e) "tags" %in% names(e)) |>  
3   map_chr("name")
```

R

```
[1] "a" "c"
```

reduce() and accumulate()

`reduce()` collapses a list to one value by applying a two-argument function cumulatively, `f(f(f(x1, x2), x3), x4)`; `accumulate()` does the same but keeps the intermediate results.

```
1 reduce(1:5, `+`)
```

R

```
[1] 15
```

```
1 accumulate(1:5, `+`)
```

R

```
[1] 1 3 6 10 15
```

```
1 x = list(1:3, 2:4, 3:5)
2 reduce(x, intersect)
```

R

```
[1] 3
```

```
1 dfs = list(
2   tibble(id = 1:2, a = 1:2),
3   tibble(id = 1:2, b = 3:4),
4   tibble(id = 1:2, c = 5:6)
5 )
6 reduce(dfs, left_join, by = "id")
```

R

```
# A tibble: 2 × 4
   id     a     b     c
<int> <int> <int> <int>
1     1     1     3     5
2     2     2     4     6
```

To stack a list of data frames by row use `list_rbind()` (`pd.concat()` or `pl.concat()` in Python), not `reduce()` with

Functional tools in Python

map() and filter()

Python's built-in `map(f, iterable)` and `filter(pred, iterable)` are the analogs of purrr's `map()` and `keep()`. Both return lazy iterators - nothing is computed until the values are consumed (`list()`, a `for` loop, `sum()`, ...) and they can be consumed only once.

```
1 m = map(lambda x: x**2, range(5))
2 m
```

py

<map object at 0x11a3764c0>

```
1 list(m)
```

py

[0, 1, 4, 9, 16]

```
1 list(m)
```

py

[]

```
1 list(filter(
2     lambda x: x % 2 == 0, range(10)
3 ))
```

py

[0, 2, 4, 6, 8]

```
1 list(map(pow, [1, 2, 3], [4, 5, 6]))
```

py

[1, 32, 729]

`map()` with several iterables is the analog of `map2()` / `pmap()`. There are no typed variants - a list can hold anything, so

Comprehensions

Python style prefers a comprehension to `map()` and `filter()` - transformation and filter in one expression, no `lambda`. The same syntax builds dicts and sets, and in parentheses a generator (next slide), which `sum()` and `any()` consume below.

```
1 [x**2 for x in range(5)]
```

py

```
[0, 1, 4, 9, 16]
```

```
1 [x**2 for x in range(10) if x % 2 == 0]
```

py

```
[0, 4, 16, 36, 64]
```

```
1 [a + b for a, b in zip("ab", "cd")]
```

py

```
['ac', 'bd']
```

```
1 {x: x**2 for x in range(4)}
```

py

```
{0: 0, 1: 1, 2: 4, 3: 9}
```

```
1 {x % 3 for x in range(10)}
```

py

```
{0, 1, 2}
```

```
1 sum(x**2 for x in range(5))
```

py

```
30
```

```
1 any(x > 3 for x in range(5))
```

py

```
True
```

Generators

A generator function uses `yield` instead of `return`. Calling it runs nothing; it returns a generator that produces one value each time it is asked, pausing after each `yield`. A generator expression, `(f(x) for x in xs)`, is the one-line form.

```
1 def countdown(n):  
2     while n > 0:  
3         yield n  
4         n -= 1
```

```
1 g = countdown(3)  
2 type(g)
```

<class 'generator'>

```
1 next(g)
```

3

```
1 next(g)
```

2

```
1 next(g)
```

1

```
1 next(g)
```

StopIteration

```
1 list(countdown(3))
```

[3, 2, 1]

```
1 [x**2 for x in countdown(3)]
```

[9, 4, 1]

Laziness

Creating a generator does no work; each `next()` runs it only as far as the following `yield`. So a generator can be infinite, stop early, or read a file one line at a time, and chained generators form a pipeline.

```
1 def loud(xs):  
2     for x in xs:  
3         print("yielding", x)  
4         yield x  
5  
6 g = loud([1, 2, 3])
```

```
1 next(g)
```

```
yielding 1  
1
```

```
1 list(g)
```

```
yielding 2  
yielding 3  
[2, 3]
```

```
1 def naturals():  
2     n = 1  
3     while True:  
4         yield n  
5         n += 1
```

```
1 from itertools import islice  
2 squares = (n**2 for n in naturals())  
3 list(islice(squares, 5))
```

```
[1, 4, 9, 16, 25]
```

```
1 next(s for s in squares if s > 1000)
```

```
1024
```

reduce() and friends

Reductions live in the standard library - `functools.reduce()` for a general fold, `itertools.accumulate()` for the running version, and `sum()`, `min()`, `max()`, `any()`, `all()` for common cases. `operator` has function versions of the operators.

```
1 from functools import reduce
2 from itertools import accumulate
3 import operator
```

```
1 reduce(operator.add, range(1, 6))
```

15

```
1 list(accumulate(range(1, 6)))
```

[1, 3, 6, 10, 15]

```
1 reduce(
2     lambda a, b: a + b,
3     [[1, 2], [3], [4, 5]]
4 )
```

[1, 2, 3, 4, 5]

```
1 list(accumulate([1, 3, 2, 5, 4], max))
```

[1, 3, 3, 5, 5]

Rectangling

GitHub repositories

The GitHub API's `/orgs/{org}/repos` endpoint lists an organization's repositories as an array of objects, one per repository, each with the same 83 fields (abridged here):

```
1  [  
2    {  
3      "id": 19438,  
4      "name": "ggplot2",  
5      "owner": { "login": "tidyverse", "type": "Organization", ... },  
6      "stargazers_count": 7002,  
7      "language": "R",  
8      "license": { "key": "other", "name": "Other", ... },  
9      "topics": ["data-visualisation", "r", "visualisation"],  
10     ...  
11   },  
12   ...  
13   {  
14     "name": "nycflights13",  
15     "license": null,  
16     "topics": [],  
17     ...  
18   },  
19   ...  
20 ]
```

Reading JSON

`jsonlite::read_json()` and Python's `json.load()` keep the structure as is: the array becomes a list, each object a named list (R) or dict (Python) with nested objects staying nested.

```
1 repos = jsonlite::read_json(  
2   "data/tidyverse_repos.json"  
3 )
```

R

```
1 length(repos)
```

R

```
[1] 46
```

```
1 repos[[1]]$name
```

R

```
[1] "ggplot2"
```

```
1 repos[[1]]$license$name
```

R

```
[1] "Other"
```

```
1 import json  
2 path = "data/tidyverse_repos.json"  
3 with open(path) as f:  
4     repos = json.load(f)
```

py

```
1 len(repos)
```

py

```
46
```

```
1 repos[0]["name"]
```

py

```
'ggplot2'
```

```
1 repos[0]["license"]["name"]
```

py

```
'Other'
```

Extracting columns - purrr

A tidy version has one row per repository and one column per variable. The direct approach builds each column with a `map_*()`.

```
1 tibble(  
2   name      = map_chr(repos, "name"),  
3   stars     = map_int(repos, "stargazers_count"),  
4   n_topics  = map_int(repos, \(r) length(r$topics))  
5 )
```

```
# A tibble: 46 × 3  
  name      stars n_topics  
  <chr>    <int>   <int>  
1 ggplot2    7002     3  
2 lubridate   805     3  
3 stringr    672     3  
4 dplyr     5071     3  
5 readr     1041     4  
# i 41 more rows
```

Extracting columns - Python

The same idea with one comprehension per column. `pd.DataFrame()` and `pl.DataFrame()` accept a dict of lists like this, or a list of dicts, one per row.

```
1 pd.DataFrame({
2     "name":      [r["name"] for r in repos],
3     "stars":     [r["stargazers_count"] for r in repos],
4     "n_topics": [len(r["topics"]) for r in repos]
5 })
```

	name	stars	n_topics
0	ggplot2	7002	3
1	lubridate	805	3
..	...	...	...
44	ggbot2	38	0
45	data-dict	107	0

[46 rows x 3 columns]

Missing values

Some repositories have no license, so `license` is `NULL` in R and `None` in Python.

`NULL` cannot fill a vector slot, so `map_chr()` fails without `.default`. `None` is an okay list element, but indexing into it fails. A conditional expression gives the name when there is a license and `None` otherwise.

```
1 map_chr(repos, c("license", "name"))
```

R

```
Error in `map_chr()`:  
i In index: 8.  
Caused by error:  
! Result must be length 1, not 0.
```

```
1 [r["license"]["name"] for r in repos]
```

py

```
TypeError: 'NoneType' object is not subscriptable
```

```
1 license = map_chr(  
2   repos, c("license", "name"),  
3   .default = NA  
4 )  
5 license[7:9]
```

R

```
[1] "0ther" NA      "0ther"
```

```
1 license = [  
2   r["license"]["name"] if r["license"] else  
3   for r in repos  
4 ]  
5 license[6:9]
```

py

```
['0ther', None, '0ther']
```

Exercise 1

Using `repos` (the tidyverse GitHub repositories) with `purrr` in R and comprehensions in Python,

1. Construct a data frame with two columns: each repository's `name` and its primary `language`.
2. Find the five repositories with the most stars (`stargazers_count`).
3. Find the names of the repositories with `"r"` among their `topics`.

List columns

The other approach puts the whole list into a data frame first and takes it apart from there. A list column holds an arbitrary object per cell; polars infers a `Struct` (named fields) for dicts and a `List` for lists, pandas stores either as `object`.

```
1 (d = tibble(repo = repos))
```

```
# A tibble: 46 × 1
  repo
  <list>
1 <named list [83]>
2 <named list [83]>
3 <named list [83]>
4 <named list [83]>
5 <named list [83]>
# i 41 more rows
```

```
1 print(pd.DataFrame({"repo": repos}))
```

```
      repo
0  {'id': 19438, 'node_...
1  {'id': 148017, 'node...
..      ...
44 {'id': 1060311848, '...'
45 {'id': 1184566747, '...'

[46 rows x 1 columns]
```

```
1 print(pl.DataFrame({"repo": repos}))
```

```
shape: (46, 1)
```

repo
struct[83]
{19438, "MDEw0lJlcG9zaXRvcnkxOT..."}
{148017, "MDEw0lJlcG9zaXRvcnkxN..."}
{365649, "MDEw0lJlcG9zaXRvcnkzN..."}
...
{1060311848, "R_kgD0PzMTKA", "gg..."}
{1184566747, "R_kgD0RpsN2w", "da..."}

unnest_wider()

`unnest_wider()` makes each element of the named lists its own column - for list columns where every element has the same names. Types are inferred from the values, `NULL` becomes `NA`, and a nested object stays as a list column.

```
1 d |> unnest_wider(repo)
```

```
# A tibble: 46 × 83
   id node_id name full_name private owner html_url description fork
   <int> <chr> <chr> <chr> <lgl> <list> <chr> <chr> <lgl>
1 1.94e4 MDEw0l... ggpl... tidyvers... FALSE <named list> https:/... An impleme... FALSE
2 1.48e5 MDEw0l... lubr... tidyvers... FALSE <named list> https:/... Make worki... FALSE
3 3.66e5 MDEw0l... stri... tidyvers... FALSE <named list> https:/... A fresh ap... FALSE
4 6.43e6 MDEw0l... dplyr tidyvers... FALSE <named list> https:/... dplyr: A g... FALSE
5 1.17e7 MDEw0l... readr tidyvers... FALSE <named list> https:/... Read flat ... FALSE
# i 41 more rows
# i 74 more variables: url <chr>, forks_url <chr>, keys_url <chr>,
# collaborators_url <chr>, teams_url <chr>, hooks_url <chr>,
# issue_events_url <chr>, events_url <chr>, assignees_url <chr>,
# branches_url <chr>, tags_url <chr>, blobs_url <chr>, git_tags_url <chr>,
# git_refs_url <chr>, trees_url <chr>, statuses_url <chr>,
# languages_url <chr>, stargazers_url <chr>, contributors_url <chr>, ...
```

Nested objects

`license` is a list column of named lists (or `NULL`), so a second `unnest_wider()` spreads it. `names_sep` adds a prefix, since the license's `name` and `url` would collide with the repository's; the `NULL`s become rows of `NA`.

```
1 (lic = d |> unnest_wider(repo) |> select(name, license) |> slice(7:9))
```

```
# A tibble: 3 × 2
  name      license
<chr>      <list>
1 tidyr    <named list [5]>
2 nycflights13 <NULL>
3 rvest     <named list [5]>
```

```
1 lic |>
2   unnest_wider(license, names_sep = "_")
```

```
# A tibble: 3 × 6
  name      license_key license_name license_spdx_id license_url license_node_id
<chr>      <chr>          <chr>          <chr>          <lgl>      <chr>
1 tidyr    other          Other          NOASSERTION    NA          MDc6TG1jZW5zZT...
2 nycfligh... <NA>          <NA>          <NA>          NA          <NA>
3 rvest     other          Other          NOASSERTION    NA          MDc6TG1jZW5zZT...
```

pandas - json_normalize()

pandas has no `unnest_wider()`, but the same workflow is three steps: `pd.json_normalize()` spreads the dicts in a column into a data frame (`max_level=0` leaves nested dicts as they are), `join()` attaches those columns, and `drop()` removes the original.

```
1 df = pd.DataFrame({"repo": repos})
2 wide = (
3     df
4     .join(pd.json_normalize(df["repo"], max_level=0))
5     .drop(columns="repo")
6 )
7 wide[["name", "language", "license"]]
```

	name	language	license
0	ggplot2	R	{'key': 'other', 'na...
1	lubridate	R	{'key': 'other', 'na...
..	...	...	...
44	ggbot2	R	{'key': 'other', 'na...
45	data-dict	Rust	None

[46 rows x 3 columns]

`pd.DataFrame(repos)` and `pl.DataFrame(repos)` build this wide frame from the list of dicts in one step; the workflow

pandas - nested objects

`license` is a column of dicts (or `None`), so the same three steps spread it: `json_normalize()` the column, `add_prefix()` in place of `names_sep` so the license's `name` and `url` do not collide with the repository's, then `join()` and `drop()`. The `Nones` become rows of `NaN`.

```
1 lic = wide[["name", "license"]]
2 lic.iloc[6:9]
```

	name	license
6	tidyr	{'key': 'other', 'na...
7	nycflights13	None
8	rvest	{'key': 'other', 'na...

```
1 lic_wide = (lic
2     .join(pd.json_normalize(lic["license"]).add_prefix("license_"))
3     .drop(columns="license")
4 )
5 print(lic_wide.iloc[6:9].to_string())
```

	name	license_key	license_name	license_spdx_id	license_url	license_node_id
6	tidyr	other	Other	NOASSERTION	NaN	MDc6TGljZW5zZTA=
7	nycflights13	NaN	NaN	NaN	NaN	NaN
8	rvest	other	Other	NOASSERTION	NaN	MDc6TGljZW5zZTA=

polars - unnest()

polars' `unnest()` is `unnest_wider()` for a `Struct` column: one call spreads the fields of `repo` into the 83 columns, and the nested `license` is itself a struct (or `null`).

```
1 repos_pl = pl.DataFrame({"repo": repos}).unnest("repo")
2 repos_pl.select("name", "language", "license")
```

py

shape: (46, 3)

name	language	license
str	str	struct[5]
"ggplot2"	"R"	{"other","Other","NOASSERTION",null,"MDc6TGljZW5zZTA="}
"lubridate"	"R"	{"other","Other","NOASSERTION",null,"MDc6TGljZW5zZTA="}
"stringr"	"R"	{"other","Other","NOASSERTION",null,"MDc6TGljZW5zZTA="}
...	...	...
"ggbot2"	"R"	{"other","Other","NOASSERTION",null,"MDc6TGljZW5zZTA="}
"data-dict"	"Rust"	null

polars - nested objects

`unnest()` errors on name collisions (`name`, `url`, ...), so prefix the struct's fields first. `struct.field()` extracts a single field instead, renamed with `alias()` here since the repository already has a `name` column. `pl.json_normalize()` flattens nested dicts like the pandas function.

```
1 repos_pl.with_columns(  
2     pl.col("license")  
3     .name.prefix_fields("license_")  
4 ).unnest(  
5     "license"  
6 ).select(  
7     "name", "license_name"  
8 )
```

py

shape: (46, 2)

name	license_name
str	str
"ggplot2"	"Other"
"lubridate"	"Other"
"stringr"	"Other"
...	...
"ggbot2"	"Other"
"data-dict"	null

```
1 repos_pl.select(  
2     "name",  
3     pl.col("license")  
4     .struct.field("name")  
5     .alias("license")  
6 )
```

py

shape: (46, 2)

name	license
str	str
"ggplot2"	"Other"
"lubridate"	"Other"
"stringr"	"Other"
...	...
"ggbot2"	"Other"
"data-dict"	null

unnest_longer() and explode()

When an element is an array of values, the tidy shape is one row per value:

`unnest_longer()` in tidy, `explode()` in pandas and polars. Each repository has zero or more topics,

```
1 d |>
2   unnest_wider(repo) |>
3   select(name, topics) |>
4   unnest_longer(topics)
```

```
# A tibble: 96 × 2
  name      topics
<chr>    <chr>
1 ggplot2  data-visualisation
2 ggplot2  r
3 ggplot2  visualisation
4 lubridate date
5 lubridate date-time
# i 91 more rows
```

```
1 (wide[["name", "topics"]]
2   .explode("topics")
3 )
```

	name	topics
0	ggplot2	data-visualisation
0	ggplot2	r
..	...	...
44	ggbot2	NaN
45	data-dict	NaN

[112 rows x 2 columns]

```
1 (repos_pl
2   .select("name", "topics")
3   .explode("topics")
4 )
```

shape: (112, 2)

name	topics
str	str
"ggplot2"	"data-visualisation"
"ggplot2"	"r"
"ggplot2"	"visualisation"
...	...
"ggbot2"	null
"data-dict"	null

16 repositories have no topics - tidy drops them unless `keep_empty = TRUE`, pandas keeps as NaNs, and polars' uses

hoist()

`hoist()` pulls a few fields out of a list column by name, position, or path ([NA](#) when missing) and leaves the rest. Effective when we don't want all the columns from unnesting.

```
1 tibble(repo = repos) |>
2   hoist(
3     repo, name = "name",
4     license = c("license", "name"),
5     first_topic = list("topics", 1)
6   )
```

R

A tibble: 46 × 4

	name	license	first_topic	repo
	<chr>	<chr>	<chr>	<list>
1	ggplot2	Other	data-visualisation	<named li
2	lubridate	Other	date	<named li
3	stringr	Other	r	<named li
4	dplyr	Other	data-manipulation	<named li
5	readr	Other	csv	<named li
# i 41 more rows				

```
1 def at(x, i):
2     return x[i] if x else None
3 pd.DataFrame([
4     "name": r["name"],
5     "license": at(r["license"], "name"),
6     "first_topic": at(r["topics"], 0)
7 ] for r in repos])
```

py

	name	license	first_topic
0	ggplot2	Other	data-visualisation
1	lubridate	Other	date
..	...	...	...
44	ggbot2	Other	NaN
45	data-dict	NaN	NaN
[46 rows x 3 columns]			

General advice

- Same-named entries in every element: go wider with `unnest_wider()`, `pd.json_normalize()`, or polars' `unnest()`.
- Arrays of varying length: go longer with `unnest_longer()` or `explode()`.
- Avoid tidyr's plain `unnest()`, which expects a list column of data frames, and `unnest_auto()`, which guesses between wider and longer from the data.
- Do not rectangle what you will not use - `hoist()`, `map_*()`, and comprehensions target just the fields you need.
- Check the column types afterwards - a list (or `object`) column usually means inconsistent values: a scalar in some records, an array or missing in others.

Exercise 2

Most of the 83 fields are API URLs that you will never use. Using `tidyr` / `purrr` in R and `pandas` or `polars` in Python, tidy `repos` into a data frame with one row per repository and just the useful fields: `name`, `description`, `language`, `stargazers_count`, `forks_count`, the license name (missing where there is no license), and the number of topics. Check the column types when you are done.

Summary

Functional tools

Concept	R	Python
anonymous function	<code>\(x) x^2, function(x) x^2</code>	<code>lambda x: x**2</code>
apply to each	<code>map(x, f), lapply(x, f)</code>	<code>[f(v) for v in x], map(f, x)</code>
typed result	<code>map_dbl(), map_chr(), ...</code>	none, check it yourself
two or more inputs	<code>map2(), pmap()</code>	<code>zip(), map(f, a, b)</code>
index and value	<code>imap()</code>	<code>enumerate()</code>
filter	<code>keep(), discard()</code>	<code>[v for v in x if p(v)], filter()</code>
reduce	<code>reduce(), accumulate()</code>	<code>functools.reduce(), itertools.accumulate()</code>
extract by name	<code>map(x, "name")</code>	<code>[d["name"] for d in x], operator.itemgetter()</code>
missing element	<code>.default =</code>	<code>x["k"] if x else None, d.get("k")</code> for a missing key
side effects	<code>walk()</code>	for loop
partial application	<code>partial(f, p = 3), ... in map()</code>	<code>functools.partial(f, p=3)</code>
lazy sequence	none, vectors are eager	generator (<code>yield</code>), <code>map()</code> , <code>zip()</code>

Rectangling

Task	tidyr / purrr	pandas	polars
read JSON	<code>jsonlite::read_json()</code>	<code>json.load()</code>	<code>json.load()</code>
list column	<code>tibble(x = lst)</code>	<code>pd.DataFrame({"x": lst}),</code> <code>object</code>	<code>pl.DataFrame({"x":</code> <code>lst}), Struct</code>
object to columns	<code>unnest_wider()</code>	<code>pd.json_normalize()</code>	<code>unnest(),</code> <code>pl.json_normalize()</code>
array to rows	<code>unnest_longer()</code>	<code>explode()</code>	<code>explode()</code>
a few fields	<code>hoist(), map_chr(x, c("a",</code> <code>"b"))</code>	<code>[d["a"]["b"] for d in x]</code>	<code>struct.field()</code>
missing values	<code>NULL, NA</code> after unnesting	<code>None, NaN</code>	<code>null</code>

Takeaways

- Functions are values in both languages - pass them to other functions, store them, fix some of their arguments with `partial()`, and write short ones inline with `\(x)` or `lambda`.
- purrr's `map_*()` family replaces `lapply()` / `sapply()` with type-stable iteration, plus shorthands for extraction, multiple inputs, filtering, and reducing. Python has `map()`, `filter()`, `functools.reduce()`, and lazy generators, but a comprehension is the idiom.
- JSON becomes nested lists (R) or dicts and lists (Python). Rectangling is extracting fields and unnesting - wider for objects, longer for arrays - until each row is an observation and each column a variable.
- Prefer targeted extraction (`map_*()`, `hoist()`, a comprehension) over unnesting everything, and check the column types when you are done.

Example

Rectangling `data/discog.json`